CAREFLOW – CLINICAL PATHWAY PROCESS MINING

FINAL PROJECT REPORT

Infotact Solutions – Data Analytics Internship

Group number: 4

Batch: 20.1

Team members: -

Anagha S Pillai (Team Leader)

Vaishnavi Dipak Bhagat

Yashaswini N P

ABSTRACT

CareFlow is an end-to-end healthcare process analytics project developed to understand how patients move through a hospital workflow. The project starts with timestamped event-log generation and cleaning, then applies exploratory analysis, bottleneck and waiting-time analysis, PM4Py process discovery, conformance checking and root-cause investigation. The final solution extends the original process-mining workflow with BigQuery and dbt so that transformed analytics tables can directly support the Power BI dashboard. The final reproducible dataset contains 200 patient cases and 878 events. The expected pathway is Triage → X-Ray → Doctor Consult → Discharge. The main observed loop is Triage → X-Ray → Triage, affecting 78 cases (39.0%). Therefore, 122 cases (61.0%) are conformant and 78 (39.0%) are deviating. Python and BigQuery/dbt calculations were cross-validated, and the final Power BI dashboard was connected to the BigQuery analytics layer and enhanced with patient-level drillthrough.

1. Project Overview

CareFlow – Clinical Pathway Process Mining is a healthcare process analytics project designed to understand and analyze patient journeys through hospital workflows.

The project uses timestamped hospital event logs to study the sequence of activities performed during patient journeys. Process mining and data analytics techniques are used to discover the actual workflow, identify repeated activities, detect potential bottlenecks, analyze waiting and transition times, and compare actual patient pathways with an expected workflow.

2. Problem Statement

Hospital workflows involve multiple activities such as registration, triage, consultation, diagnostic tests, review, pharmacy, and discharge.

Traditional dashboards can provide numerical information such as patient counts and waiting times, but they may not clearly represent the actual sequence of activities followed by patients.

Patients may experience:

- Repeated activities
- Long waiting times
- Process loop-backs
- Different patient pathways
- Deviations from the expected workflow
- Inefficient transitions between activities

One of the observed patterns in the project is:

Triage

↓

X-Ray

↓

Triage

The repeated Triage activity represents a process deviation that requires further investigation.

CareFlow aims to use process mining and data analytics to identify, quantify, and visualize such process patterns.

3. Objectives

- Generate realistic timestamped patient event logs.
- Clean, standardize and validate event data.
- Perform EDA on patients, activities, sequences and waiting times.
- Identify repeated activities and potential bottlenecks.
- Discover the actual workflow using PM4Py.
- Generate DFG and Petri Net process models.
- Compare actual journeys with the expected pathway through conformance checking.
- Investigate a simulated root-cause field, Missing_Paperwork.
- Build a BigQuery + dbt analytics pipeline with automated data-quality checks.
- Cross-validate Python findings against BigQuery/dbt outputs.
- Connect the analytics layer to Power BI and provide patient-level drillthrough.

4. Final End-to-End Workflow

Hospital Event Data → Python Generation → Cleaning → EDA → Bottleneck/Waiting-Time Analysis → PM4Py → DFG/Petri Net → Conformance → Root-Cause Investigation → BigQuery Raw → dbt Staging → dbt Analytics → Cross-Validation → Power BI → Patient Drill through → Final Insights

5. Technology Stack

Technology	Use in CareFlow
Python	Event generation, processing and validation
Pandas	Cleaning, EDA, sequence analysis and independent validation
Matplotlib	EDA visualizations

PM4Py	Process discovery and conformance analysis
Graphviz	Process-map visualization
BigQuery	Cloud storage and query layer for event analytics
dbt	SQL transformations, analytics models and data-quality tests
Power BI	Interactive KPI dashboard, process analysis and drillthrough
Git/GitHub	Version control and collaboration

6. Dataset and Data Preparation

The event log represents hospital patient journeys. The core fields are Case_ID, Activity_Name and Timestamp. Missing_Paperwork is included as a simulated patient-level attribute for the root-cause proof of concept.

Field	Meaning
Case_ID	Unique patient process case
Activity_Name	Hospital activity performed
Timestamp	Time at which the activity occurred
Missing_Paperwork	Simulated indicator used to test a root-cause hypothesis

Final stable dataset:

Metric	Value
Patient cases	200
Events	878
Cleaned rows	878
Rows removed	0
Random seed	42

7. Weekly Project Progress

Week 1 – Project Setup, Data Preparation and Initial Analysis

- Set up the project repository, branches, documentation and tracking workflow.
- Developed the Python event generator and simulated hospital patient journeys.
- Generated and cleaned the initial event log using Python and Pandas.
- Performed EDA, activity analysis and waiting-time analysis.
- Identified the Triage → X-Ray → Triage loop as the main process deviation.
- Created the data requirements and data dictionary.
- Installed and configured PM4Py for process-mining analysis.
- Completed initial README, project documentation and fresh-clone testing.

Week 2 – Process Mining, Conformance and Mid-Review Preparation

- Implemented formal PM4Py process discovery.
- Generated the Directly-Follows Graph (DFG) and Petri Net process model.
- Completed conformance checking against the expected clinical pathway.
- Developed the initial Power BI dashboard with KPI cards, DAX measures and process-analysis visuals.
- Refined dashboard terminology and layout based on team feedback.
- Documented process-map findings and the initial BigQuery/dbt scope decision.
- Performed consistency and validation checks across process-mining outputs.

- Prepared the Mid-Review report, presentation and supporting documentation

Week 3 – Process Mining, Mid-Review & Dashboard Refinement

- Continued process-mining and conformance analysis after the initial implementation.
- Refined the Power BI dashboard based on review feedback.
- Performed consistency checks across process maps, conformance results and dashboard figures.
- Completed Mid-Review presentation and the documentation/next-steps section.
- Consolidated the technical findings and documentation for the final project phase.

Week 4 – Data Engineering, Root Cause & Final Integration

- Investigated the recurring process loop using the simulated Missing_Paperwork attribute.
- Implemented the BigQuery + dbt Raw → Staging → Analytics pipeline.
- Built the dbt analytics models: patient_events, patient_transitions, case_summary, and conformance_summary.
- Implemented and passed 15/15 automated data-quality tests.
- Corrected the repeated-activity logic in case_summary.
- Fixed the random seed to 42 and stabilized the final dataset at 878 events / 200 patients.
- Cross-validated Python vs BigQuery/dbt results, confirming 78 deviating cases (39.0%).
- Connected case_summary and conformance_summary directly to Power BI.
- Implemented and tested patient-level Drillthrough using Case_ID.
- Corrected the stale Deviating % KPI from 36.5% to the final 39.0%.
- Completed the Final Review documentation and README files.

8. Data Generation and Cleaning

- The generator simulates patient journeys with timestamps. Normal cases follow Triage → X-Ray → Doctor Consult → Discharge. Deviating cases include an additional Triage after X-Ray. Randomness was initially allowed to vary between runs; the final implementation uses random.seed(42), making the final dataset reproducible.
- The cleaning script checks missing values, validates timestamps, checks duplicates, standardizes activity values and prepares the event log for analysis.

9. Exploratory Data Analysis

- Dataset overview and structure.
- Number of patient cases and events.
- Activity frequency distribution.
- Average events per patient.
- Repeated activity analysis.
- Triage → X-Ray → Triage loop-back analysis.
- Waiting time between consecutive activities.
- Activity and transition visualizations.
- The final analysis identifies 78 loop-back cases out of 200, equal to 39.0%.

10. Bottleneck and Waiting-Time Analysis

10.1 Bottleneck Analysis

The bottleneck analysis focused on identifying repeated and potentially inefficient process patterns.

The main process pattern identified was:

Triage

↓

X-Ray

↓

Triage

Out of the 200 patient cases:

- **78 patients** experienced the Triage → X-Ray → Triage loop.
- This represents **39% of all patient cases**.

The repeated Triage pattern was therefore identified as the primary process deviation requiring further investigation.

The finding was initially identified through EDA and was subsequently examined using PM4Py process discovery and conformance checking.

10.2 Waiting-Time Analysis

Waiting-time analysis was performed to understand the time spent between consecutive activities in the patient journey.

Particular attention was given to transitions associated with the identified process loop, including:

X-Ray → Triage

The analysis also considered important transitions such as:

Triage → X-Ray

X-Ray → Triage

Triage → Doctor Consultation

Doctor Consultation → X-Ray

Waiting-time analysis provides additional information about process delays and helps complement the bottleneck analysis.

The resulting information is being incorporated into the Power BI dashboard through the **Average Wait Time** KPI and transition/waiting-time visuals.

The main process deviation is Triage → X-Ray → Triage. This pattern indicates that a subset of patients returns to Triage after diagnostic testing before continuing. It is treated as an observed process deviation and potential bottleneck; the root cause is investigated separately.

Measure	Final result
Loop-back cases	78 / 200
Loop-back rate	39.0%
Average event-to-event wait	≈13.52 minutes
Average X-Ray → Triage wait for loop cases	≈19.95 minutes

11. PM4Py Process Discovery

PM4Py was used to move from manual event-log analysis to formal process mining. The cleaned event log is converted into a process representation, allowing frequent transitions, loops and alternative behavior to be viewed systematically.

- Directly-Follows Graph (DFG)
- Petri Net process model
- Process-map visualization
- Fresh-clone verification of the PM4Py workflow

11.1 Directly-Follows Graph

The Directly-Follows Graph represents transitions between consecutive hospital activities.

It helps identify:

- Common activity transitions
- Frequently occurring process paths
- Process loops
- Repeated transitions
- Different patient activity sequences

The DFG provides a formal process-level representation of the patterns initially identified during EDA.

The following loop-back pattern was observed:

Triage

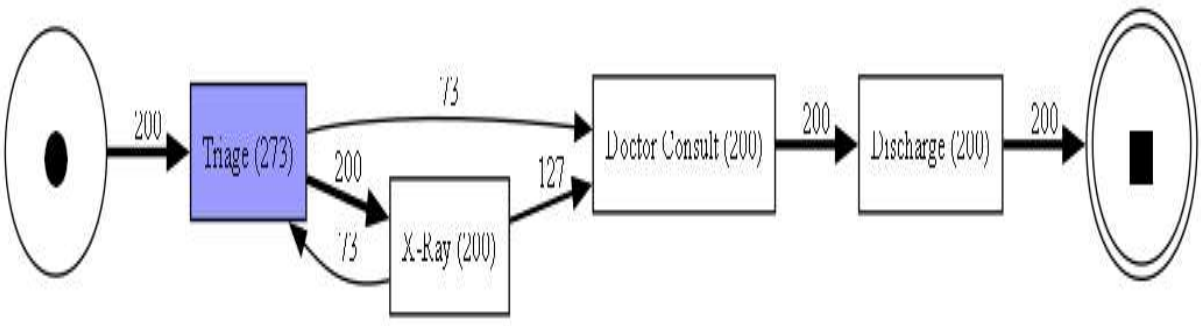
↓

X-Ray

↓

Triage

The DFG therefore provides visual evidence of the repeated process behavior in the event log.



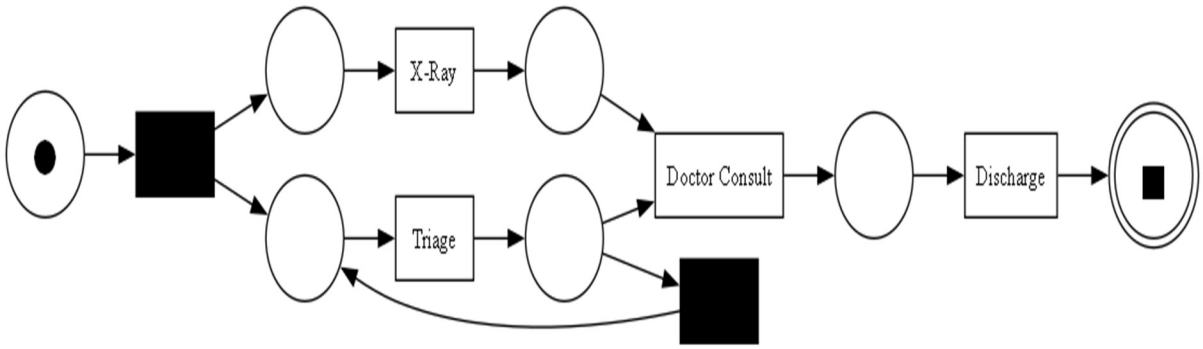
11.2 Petri Net Process Model

A Petri Net process model was generated using PM4Py.

The Petri Net provides a structured representation of:

- Activities
- Process transitions
- Possible patient paths
- Process flow relationships
- Repeated process behavior

The generated Petri Net model was successfully verified during project testing.



12. Conformance Checking

The expected pathway is Triage → X-Ray → Doctor Consult → Discharge. Each of the 200 patient cases is compared with this reference sequence. A conformant case follows the defined sequence; additional/repeated activity produces a deviating case.

Metric	Final
Total cases	200
Conformant	122 (61.0%)
Deviating	78 (39.0%)

13. Root-Cause Investigation

The process-mining stages establish what happened, but a root-cause analysis asks why the loop may occur. The project uses the simulated Missing_Paperwork attribute to test the hypothesis that incomplete paperwork may lead to a return to Triage.

Missing_Paperwork	Did not loop	Looped
False	122	0
True	0	78

The simulation therefore produces a 100% association between Missing_Paperwork=True and the loop. This is a proof-of-concept finding only: because the relationship is encoded in simulated data, it cannot be claimed as real-world clinical causation. A real hospital deployment would require actual intake/EHR evidence.

14. BigQuery and dbt Implementation

The final project extends the original local process-mining workflow with a BigQuery and dbt analytics layer. The raw event data is stored in BigQuery, then transformed into analysis-ready tables by dbt.

Layer/Model	Purpose	Rows
BigQuery raw: hospital_event_log	Central event-data storage	878
stg_hospital_event_log	Clean staging representation	878
patient_events	Analysis-ready event records	878
patient_transitions	Sequential activity transitions and durations	678

case_summary	One row per patient case and case-level process indicators	200
conformance_summary	Case-level conformance result	200

15. dbt Data Quality

The completed dbt pipeline includes 15 automated tests. These validate required fields, accepted activity values, case-level uniqueness and conformance status values across the project models.

Test result	Status
Automated tests	15/15 PASS
Errors	0
Warnings	0
Skipped	0

16. Cross-Validation

To ensure the data-engineering layer did not change the original analytical conclusion, the final Python analysis was compared with the dbt/BigQuery case-level output.

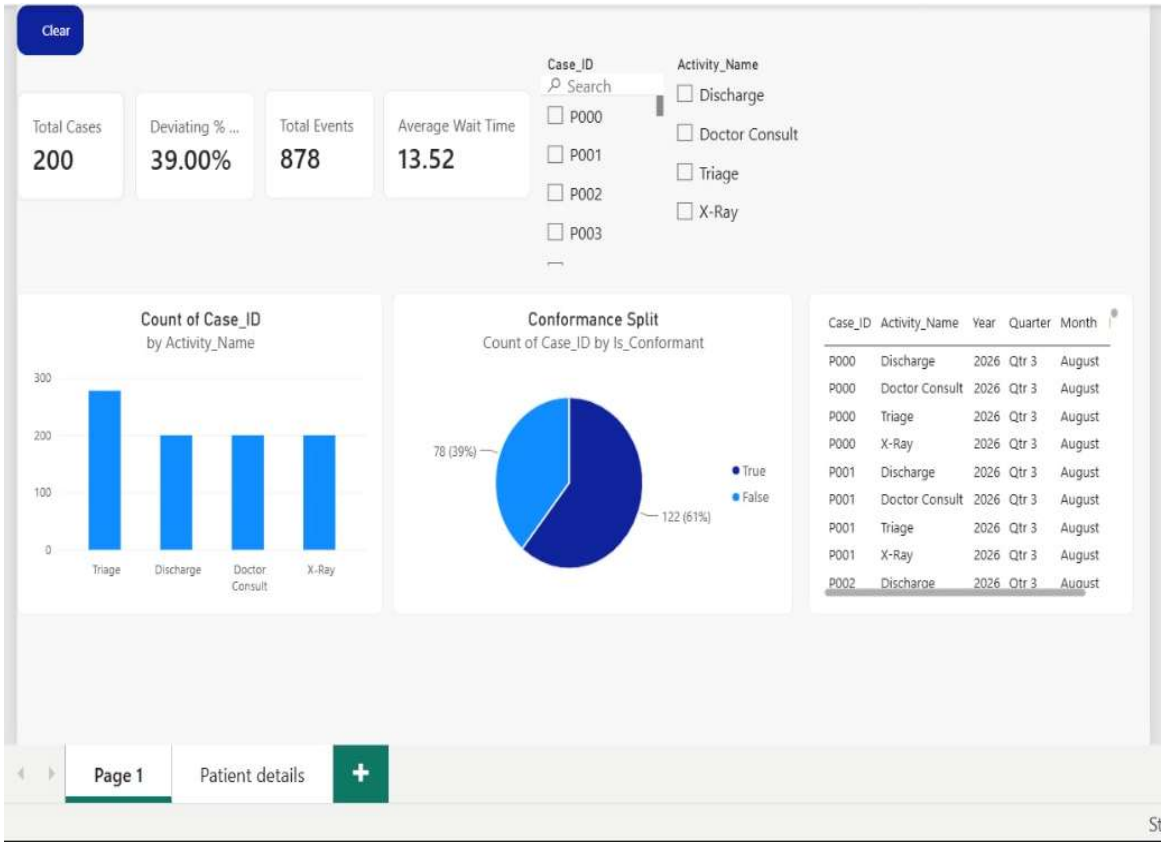
Metric	Python	BigQuery/dbt	Match
Patients	200	200	YES
Deviating cases	78	78	YES
Deviation rate	39.0%	39.0%	YES

The exact match strengthens confidence in the final result because the same central metric is reproduced independently across the Python and SQL analytics paths.

17. Power BI Dashboard

The final Power BI dashboard is connected to the completed BigQuery analytics layer so that the data-engineering work directly supports reporting. The dashboard combines high-level KPIs with process-oriented analysis.

- Total Cases and Total Events.
- Conformance % and Deviating %.
- Average Wait Time.
- Activity counts/distribution.
- Conformance split.
- Patient Details drillthrough using Case_ID.



18. Patient Drillthrough

A Patient Details page was implemented using Case_ID as the drillthrough field. Selecting a patient opens the detailed event sequence and shows Case_ID, Activity_Name and Timestamp. The feature was tested using patient P001 and verified to navigate correctly.

19. Reproducibility

The final seed-controlled generator removes run-to-run variation. random.seed(42) produces the stable 878-event dataset for 200 patients. The same final dataset is used for local analysis, BigQuery/dbt validation and Power BI reporting.

20. Final Testing

- Fresh-clone setup and dependency installation – PASS.
- Event generation and reproducibility – PASS.
- Data cleaning and validation – PASS.
- EDA – PASS.
- PM4Py process discovery – PASS.
- DFG and Petri Net generation – PASS.
- Conformance checking – PASS.
- Root-cause POC – PASS.
- BigQuery raw-table validation – PASS.
- dbt staging and analytics builds – PASS.
- 15/15 dbt data-quality tests – PASS.
- Python vs BigQuery/dbt cross-validation – PASS.
- Power BI refresh and KPI validation – PASS.
- Deviating % fix – PASS.
- Patient drillthrough – PASS.

21. Final Results

Indicator	Final result
Patients	200
Events	878
Conformant	122 (61.0%)
Deviating	78 (39.0%)
Main loop	Triage → X-Ray → Triage
Average event wait	≈13.52 min
Loop X-Ray → Triage wait	≈19.95 min
dbt tests	15/15 passed
Cross-validation	Exact match

22. Key Insights

- Process mining reveals patient sequence behavior that aggregate KPIs can hide.
- The Triage → X-Ray → Triage loop is the dominant deviation in the final simulated event log.
- 39.0% of patient cases deviate from the defined expected pathway.
- Python and BigQuery/dbt independently reproduce the same deviation result.
- Power BI converts the validated analytics into an interactive operational view.
- The root-cause analysis demonstrates a method for testing hypotheses, while the simulated nature of the field limits real-world interpretation.

23. Limitations

- The event log is simulated and does not represent real clinical EHR data.
- Missing_Paperwork is simulated and cannot establish clinical causation.
- Waiting-time values depend on the simulation's timestamp logic.
- Conformance is defined against the project's fixed reference pathway.
- Real deployment would require privacy, governance, clinical validation and operational data-quality controls.

24. Future Scope

- Validate the root-cause hypothesis with real hospital intake/EHR data.
- Support multiple clinical pathways and richer conformance rules.
- Introduce incremental/scheduled data ingestion.
- Expand anomaly detection and process-performance monitoring.
- Add department, resource and time-of-day analysis.
- Extend Power BI with more patient and operational segmentation.

25. Conclusion

CareFlow has progressed from event generation and cleaning to formal process mining, conformance checking, root-cause investigation, cloud analytics, cross-validation and interactive dashboard reporting. The final stable dataset contains 200 patients and 878 events. The central process finding is the Triage → X-Ray → Triage loop in 78 cases, resulting in 39.0% deviation. The completed BigQuery/dbt layer independently reproduces the result and now powers the Power BI dashboard. The project demonstrates an end-to-end and reproducible method for understanding not only how many patients move through a workflow, but how their individual journeys differ from the expected process.